Sztuczna inteligencja i inżynieria wiedzy
Zadanie 1

Michał Karol, Paweł Narolski, Piotr Syga

27 lutego 2026

1 Cel ćwiczenia

Niniejsze zadanie ma na celu rozwinięcie praktycznych umiejętności rozwiązywania problemów optymalizacyjnych oraz implementacji technik algorytmicznych omawianych na wykładach. Po jego ukończeniu student powinien:

1. Rozumieć podstawowe pojęcia optymalizacji,
2. Rozpoznawać wyzwania obliczeniowe związane ze znajdowaniem rozwiązań dokładnych, oraz
3. Umieć stosować podejścia praktyczne w sytuacjach ograniczonych zasobów obliczeniowych.

W szczególności student będzie potrafił zrozumieć, jak i kiedy stosuje się algorytmy aproksymacyjne i heurystyki. Ponadto zapozna się z różnymi technikami przeszukiwania heurystycznego oraz zrozumie ich zastosowanie w problemach znajdowania ścieżek.

2 Wprowadzenie teoretyczne

W niniejszej sekcji znajdują się informacje przydatne przy rozwiązywaniu postawionych problemów. Zakłada się, że po zakończeniu ćwiczenia studenci w pełni znają opisane poniżej pojęcia teoretyczne.

2.1 Definicje

Definicja 1 (Problem optymalizacyjny)**.** *Problem optymalizacyjny to zbiór ograniczeń (podanych w formie nierówności lub równości ze zmiennymi decyzyjnymi) wraz z funkcją celu. Niech S będzie zbiorem rozwiązań dopuszczalnych w rozpatrywanym problemie, tj. zbiorem takich S, które spełniają wszystkie ograniczenia, a $f(\bar{x}) : \mathbb{K}^n \to \mathbb{K}$ funkcją oceny jakości rozwiązania. Definiujemy problem minimalizacji jako znalezienie takiego $s^* \in S$, dla którego $s^* = \min_s f(s)$, a $f(s)$ jest funkcją kosztu (lub straty). Alternatywnie, problem nazywamy maksymalizacyjnym, jeśli celem jest znalezienie takiego $s^* \in S$, dla którego $s^* = \max_{s \in S} f(s)$, gdzie $f(s)$ jest funkcją użyteczności.*

Łatwo zauważyć, że każdy problem minimalizacji można przekształcić w problem maksymalizacji.

Definicja 2 (Rozwiązanie optymalne)**.** *Niech S będzie zbiorem dopuszczalnym dla problemu (minimalizacji) $\mathcal{P}$ z funkcją kosztu f. Rozwiązanie $s^\star \in S$ jest optimum globalnym* ***wtedy i tylko wtedy****, gdy $\forall_{s \in S} f(s^\star) \leq f(s)$.*

Definicja 3 (Aproksymacja)**.** *Niech $\mathrm{Opt}(x) = \min_{s \in S(x)} f(s)$. Dla algorytmu M rozwiązującego problem optymalizacyjny ($M(x) \in S(x)$), nazywamy go ε–aproksymacją dla $\varepsilon \geq 0$, wtedy i tylko wtedy, gdy dla każdego x zachodzi*

$$\frac{|f(M(X)) - \mathrm{Opt}(x)|}{\max\{\mathrm{Opt}(x), f(M(x))\}} \leq \varepsilon \ .$$

Definicja 4 (Heurystyka)**.** *Niech P będzie problemem optymalizacyjnym, a S zbiorem rozwiązań problemu P. Niech $f : S \to \mathbb{R}$ będzie funkcją kosztu. Niech s^* będzie rozwiązaniem globalnie optymalnym.*

Heurystyka H dla problemu P to funkcja $H : S \to S$ zwracająca rozwiązanie $s' \in S$ problemu P, takie że w większości przypadków rozwiązanie s jest wystarczająco dobre, co oznacza, że różnica $|s^* - s'|$ nie jest znacząca.

Heurystyki mogą wykorzystywać uproszczenia, aproksymacje i reguły empiryczne w celu zmniejszenia złożoności obliczeniowej lub zwiększenia szans na znalezienie dobrego rozwiązania w stosunkowo krótkim czasie.

Z formalnej definicji heurystyki wynika, że heurystyki nie są algorytmami dokładnymi, a jedynie metodami poszukiwania dobrego przybliżonego rozwiązania problemu. Heurystyki mogą być stosowane samodzielnie lub w połączeniu z algorytmami dokładnymi w celu poprawy wydajności i jakości znajdowanych rozwiązań. Heurystyki mogą być stosowane dla problemów nieaproksymowalnych.

Heurystyki znajdują zastosowanie między innymi w rozwiązywaniu problemu komiwojażera (TSP), gdzie celem jest znalezienie najkrótszej zamkniętej ścieżki łączącej wszystkie miasta ze zbioru V.

Definicja 5 (Problem komiwojażera (TSP))**.** *Niech $G = (V, E)$ będzie grafem, gdzie V jest zbiorem wierzchołków (miast do odwiedzenia) o rozmiarze n, a E jest zbiorem krawędzi łączących pary miast. Każda krawędź $(u, v) \in E$ ma nieujemny koszt $d(u, v)$ reprezentujący odległość między miastami u i v.*
Celem jest znalezienie najkrótszej zamkniętej ścieżki, która odwiedza każde miasto dokładnie raz, a następnie wraca do punktu startowego. Formalnie, należy znaleźć permutację $\pi = (v_1, v_2, \ldots, v_n)$ zbioru V, taką że suma kosztów krawędzi $D = d((\pi_1, \pi_2)) + d((\pi_2, \pi_3)) + \cdots + d((\pi_{n-1}, \pi_n)) + d((\pi_n, \pi_1))$ jest minimalna.

2.2 Algorytm A*

Algorytm A* jest algorytmem heurystycznym stosowanym do wyszukiwania najkrótszej ścieżki w grafie. Algorytm jest rozszerzeniem algorytmu Dijkstry, wykorzystującym dodatkowy moduł przybliżonej estymacji odległości od wierzchołka do celu. Należy zauważyć, że przy niewłaściwie zaprojektowanej heurystyce algorytm może zwracać wyniki dalekie od optimum. Algorytm został zaproponowany w 1968 roku przez Petera E. Harta, Nilsa Johna Nilssona i Bertrama Raphaela.

Algorytm A* opiera się na minimalizacji funkcji kosztu zdefiniowanej jako $f(curr, next) = g(curr, next) + h(next, end)$, gdzie $g(curr, next)$ jest funkcją kosztu przejścia do wierzchołka $next$, a $h(next, end)$ szacuje koszt przejścia od wierzchołka $next$ do wierzchołka end.

Powszechnie stosowane estymacje kosztu:

- Odległość Manhattan: $h(curr, next) = |curr.x - next.x| + |curr.y - next.y|$
- Odległość euklidesowa: $h(curr, next) = \sqrt{(curr.x - next.x)^2 + (curr.y - next.y)^2}$

Główna idea algorytmu A* polega na wykorzystaniu dwóch list: otwartej (open) – zawierającej wierzchołki, które zostały odwiedzone, ale nie wszyscy ich sąsiedzi zostali zbadani, oraz zamkniętej (closed) – gdzie wszystkie wierzchołki na liście zostały odwiedzone, podobnie jak wszyscy ich sąsiedzi.

Algorithm 1 Algorytm A*

Wejście: start, end
Wyjście: Lista wierzchołków i krawędzi tworzących znalezioną ścieżkę

```
start.g ← 0
start.h ← 0
start.f ← start.g + start.h
open ← list([start])
closed ← list()
while len(open) > 0 do
    node ← null
    node_cost ← +Inf
    for test_node in open do
        if f(test_node) < node_cost then
            node ← test_node
            node_cost ← f(test_node)
    if node == end then
        Znaleziono rozwiązanie
    open ← (open − node)
    closed ← (closed + node)
    for next_node in neighbourhood(node) do
        if next_node not in open and next_node not in closed then
            open ← (open + next_node)
            next_node.h = h(next_node, end)
            next_node.g = node.g + g(node, next_node)
            next_node.f = next_node.g + next_node.h
        else
            if next_node.g > node.g + g(node, next_node) then
                next_node.g = node.g + g(node, next_node)
                next_node.f = next_node.g + next_node.h
                if next_node in closed then
                    open ← (open + next_node)
                    closed ← (closed − next_node)
```

Algorytm Dijkstry Algorytm Dijkstry jest algorytmem wyszukiwania najkrótszej ścieżki w grafie ważonym z nieujemnymi wagami i jednym źródłem. Algorytm rozpoczyna się od zbioru najkrótszych odległości od źródła do wierzchołków i aktualizuje go w miarę odwiedzania kolejnych wierzchołków.
Niech $G = (V, E)$ będzie grafem ważonym ze źródłem s, zbiorem wierzchołków V i zbiorem krawędzi E. Niech $w : E \to \mathbb{R}$ będzie funkcją kosztu przejścia krawędzi. Dla każdego $v \in V$ niech $d(v)$ oznacza koszt najkrótszej ścieżki od s do v, a $p(v)$ będzie funkcją zwracającą poprzednik wierzchołka v na aktualnie najkrótszej ścieżce od s do v.

Algorytm Dijkstry działa w następujący sposób:

1. Zainicjalizuj $d(s) = 0$ oraz $d(v) = \infty$ dla każdego $v \in V \setminus \{s\}$.
2. Utwórz zbiór Q zawierający wszystkie wierzchołki grafu G.
3. Dopóki Q nie jest pusty, wykonuj:
   (a) Wybierz wierzchołek $u \in Q$ o najmniejszym $d(u)$ i usuń go z Q
   (b) Dla każdego v takiego, że $\exists_{(u,\, v) \in E}\ d(v) > d(u) + w(u,\, v)$ zaktualizuj $d(v) = d(u) + w(u,\, v)$ oraz $p(v) = u$
4. Zwróć d i p

Zbiór Q reprezentuje zbiór wierzchołków, które nie zostały jeszcze odwiedzone, a więc z jeszcze niezaktualizowaną odległością od s. W kroku 3a wybierany jest wierzchołek z Q o najmniejszej wartości $d(u)$ i dodawany do zbioru wierzchołków z najkrótszą odległością od źródła. Ponadto aktualizowane są najkrótsze odległości dla wszystkich sąsiadów wierzchołka u, jeśli $d(u) + w(u, v)$ jest mniejsze niż dotychczas zapisane $d(v)$. W ten sposób algorytm znajduje najkrótszą ścieżkę od s do każdego wierzchołka w grafie.

2.3 Przeszukiwanie lokalne i Tabu Search

Przeszukiwanie lokalne Algorytm przeszukiwania lokalnego jest metaheurystyką polegającą na iteracyjnym przeszukiwaniu sąsiedztwa bieżącego rozwiązania w celu znalezienia lepszego rozwiązania.

Niech P oznacza problem optymalizacyjny, a s bieżące rozwiązanie. Niech $N(s)$ będzie sąsiedztwem s, tj. zbiorem rozwiązań, które można uzyskać przez zmianę ustalonej liczby parametrów (np. innej wartości jednej ze zmiennych) bieżącego rozwiązania s. Niech $f(s)$ oznacza wartość funkcji celu dla rozwiązania s.

Algorytm rozpoczyna się od wygenerowania rozwiązania początkowego s_0. Następnie w każdej iteracji algorytm generuje nowe rozwiązanie $s' \in N(s)$. Jeśli $f(s') < f(s)$, algorytm akceptuje nowe rozwiązanie s' i ustawia je jako bieżące rozwiązanie s. W przeciwnym razie odrzuca nowe rozwiązanie i kontynuuje przeszukiwanie sąsiedztwa. Przeszukiwanie kończy się po spełnieniu kryterium stopu, które jest jednym z parametrów heurystyki (np. maksymalna liczba iteracji, maksymalna liczba iteracji bez aktualizacji s). Algorytm przeszukiwania lokalnego jest podatny na utknięcie w optimum lokalnym funkcji celu.

Tabu Search Algorytm Tabu Search jest metaheurystyką polegającą na iteracyjnym przeszukiwaniu sąsiedztwa bieżącego rozwiązania z uwzględnieniem zbioru ruchów niedozwolonych (tabu). W porównaniu z przeszukiwaniem lokalnym wymaga dodatkowej pamięci, ale eliminuje ponowne badanie tych samych (lub podobnych) rozwiązań, co pozwala wydostać się z optimum lokalnego.

Główne kroki algorytmu Tabu Search:

1. Generujemy rozwiązanie początkowe s_0.
2. Ustawiamy bieżące rozwiązanie s jako s_0, zbiór tabu jako $T = \emptyset$ oraz najlepsze znane rozwiązanie jako $s^* = s$.
3. Definiujemy sąsiedztwo $N(s)$.
4. Dopóki nie spełnimy warunku stopu:
   (a) Generujemy sąsiedztwo bieżącego rozwiązania $N(s)$.
   (b) Badamy całe sąsiedztwo lub (deterministycznie) próbkujemy $N(s)$.
   (c) Wybieramy najlepszego sąsiada, takiego że $s_i \in N(s) \setminus T$.
   (d) $T = T \cup N(s)$
   (e) Ustawiamy $s = s_i$, jeśli $f(s) \leq f(s^*)$, to ustawiamy $s^* = s$.
5. Zwracamy rozwiązanie s^*.

Można zauważyć, że w zależności od rozpatrywanego problemu optymalizacyjnego P, algorytm Tabu Search powinien być dostosowany do specyficznych warunków zadania. Przykładem jest czasochłonność przeszukiwania $N(s)$, które np. dla punktów w kuli o promieniu r jest nieskończone. W takim przypadku przeprowadza się próbkowanie (np. deterministyczne lub metodą Monte Carlo) rozwiązań leżących w sąsiedztwie s. Ponadto łatwo zauważyć, że tablica zabronionych rozwiązań T rośnie z liczbą kroków, wymagając coraz więcej pamięci. Podstawową metodą rozwiązania tego problemu jest ustalenie rozmiaru tablicy $t = |T|$ i traktowanie jej jako kolejki FIFO, w której w przypadku przepełnienia usuwany jest najstarszy wpis. Wybór t powinien zależeć od cech $N(s)$, ponieważ zbyt małe t dla dużych sąsiedztw ułatwia tworzenie cykli o długości większej niż t, powodując ponowne odwiedzanie tych samych rozwiązań, a więc problemy podobne do przeszukiwania lokalnego. Z drugiej strony, zbyt duże t dla małych sąsiedztw skutkuje zużyciem nadmiarowej pamięci i brakiem usuwania rozwiązań z T. W celu dopuszczenia rozwiązań z T, które wskutek próbkowania trafiły do tablicy zabronionych rozwiązań bez sprawdzenia wartości funkcji użyteczności, można zastosować funkcję (lub tablicę) aspiracji A, która określa, że jeśli rozwiązanie jest wystarczająco dobre, to jest rozpatrywane jako możliwe rozwiązanie pomimo jego obecności w T.

Warto zauważyć, że nie zawsze musimy przechowywać w T listę kompletnych rozwiązań. Wykorzystując specyfikę problemu (lub reprezentując zbiór rozwiązań jako n-wymiarowy sześcian), możemy modyfikować T:

- możemy zabraniać wierzchołków – zamiast zabraniania całego rozwiązania, zabraniamy ustalonej wartości na jednej ze współrzędnych, np. w TSP zabraniamy wszystkich rozwiązań, które mają miasto j na pozycji i lub zabraniamy zmiany pozycji miasta i.
- zabraniamy krawędzi – to podejście zabrania ruchów wewnątrz ustalonej płaszczyzny (dwie lub więcej współrzędnych), np. dla TSP w 2-opt zabrania usunięcia jednej z krawędzi.
- zabraniamy krawędzi z jednym z wierzchołków – zabraniamy ruchu wzdłuż ustalonej prostej (dla dwóch współrzędnych) lub wewnątrz ustalonej płaszczyzny (więcej współrzędnych), np. dla TSP zabraniamy wszystkich krawędzi wychodzących z ustalonego wierzchołka i.

Poniżej przedstawiono przykład pseudokodu wykorzystującego Tabu Search do rozwiązywania problemu komiwojażera.

Algorithm 2 Algorytm Knoxa: Tabu Search dla TSP

```
k ← 0
losowo wybierz rozwiązanie początkowe s
s* ← s
T ← ∅
while k < STEP_LIMIT do
    i ← 0
    while i < OP_LIMIT do
        zdefiniuj N(s), A
        wybierz najlepsze s' ∈ (N(s) \ T) ∪ A
        zaktualizuj T
        if D(s') < D(s) then
            zaktualizuj optimum lokalne s ← s'
        i++
    k++
    if D(s) < D(s*) then
        s* ← s
```

Algorytm Knoxa pozwala rozpatrywać sąsiednie rozwiązanie znajdujące się na liście tabu, jeśli jest ono wystarczająco dobre. W tym celu wykorzystuje tablicę aspiracji (A).

Przykładem tablicy aspiracji może być implementacja pamiętająca historię k ostatnich kroków algorytmu. W takim przypadku, oprócz tablicy T, przechowujmy historię H opisującą liczbę modyfikacji zmiennej w h ostatnich krokach. Ustalmy $0 < \varepsilon < 1$ i $A_i = f(s_i) + \varepsilon(k - H(i))$. Jeśli $f(s) > A_i$, wybieramy sąsiada s_i.

3 Materiały

Dane rozkładowe dostarczane są w formacie **GTFS** (General Transit Feed Specification), opublikowane przez Koleje Dolnośląskie S. A., obowiązujące od 3 marca 2026 do 12 grudnia 2026 roku. Zbiór danych jest dystrybuowany jako zestaw plików tekstowych rozdzielanych przecinkami (`.txt`), z których każdy opisuje inny aspekt sieci transportowej. Łącznie pliki te w pełni opisują topologię, rozkład jazdy oraz wzorce kursowania pociągów sieci kolejowej Kolei Dolnośląskich.

Poniżej znajduje się szczegółowy opis każdego pliku i jego pól. Pełną specyfikację formatu GTFS można znaleźć w oficjalnej dokumentacji pod adresem `https://gtfs.org/schedule/reference/`. Należy zauważyć, że po zaimplementowaniu obsługi formatu GTFS możliwe będzie wykorzystanie danych od innych przewoźników z całego świata.

3.1 `agency.txt` – Informacje o przewoźniku

Zawiera metadane dotyczące operatora transportowego. W tym zbiorze danych występuje jeden przewoźnik: Koleje Dolnośląskie (identyfikator agencji: 279).

- `agency_id` – unikalny identyfikator przewoźnika
- `agency_name` – pełna nazwa przewoźnika
- `agency_url` – adres URL strony internetowej przewoźnika
- `agency_timezone` – strefa czasowa, w której działa przewoźnik (np. `Europe/Warsaw`)
- `agency_phone` – numer telefonu kontaktowego
- `agency_email` – adres e-mail kontaktowy
- `agency_fare_url` – adres URL do zakupu biletów

3.2 `routes.txt` – Definicje tras (57 tras)

Definiuje trasy (linie) obsługiwane przez przewoźnika. Każda trasa odpowiada nazwanej linii kolejowej (np. D1, D10, D66). Pojedyncza trasa może mieć wiele kursów realizowanych o różnych porach i w różne dni.

- `route_id` – unikalny identyfikator trasy
- `agency_id` – odwołanie do przewoźnika obsługującego trasę (klucz obcy do `agency.txt`)
- `route_short_name` – krótka, publiczna nazwa trasy (np. `D1`, `D66`)
- `route_long_name` – dłuższa opisowa nazwa trasy (używana, gdy `route_short_name` jest puste, np. `KD Premium Nadmorski`)
- `route_type` – typ transportu; wartość `2` oznacza kolej
- `route_color` – heksadecymalny kod koloru do wyświetlania trasy (np. `F6C32F`)
- `route_text_color` – heksadecymalny kolor tekstu dla etykiet na tle koloru trasy

3.3 `stops.txt` – Lokalizacje przystanków (1 108 wpisów)

Opisuje wszystkie przystanki (stacje i perony) w sieci. Każdy przystanek posiada współrzędne geograficzne w układzie WGS84 i może reprezentować stację nadrzędną (`location_type` = `1`) lub konkretny peron w obrębie stacji (`location_type` = `0`). Wpisy peronów odwołują się do stacji nadrzędnej za pomocą pola `parent_station`.

- `stop_id` – unikalny identyfikator przystanku lub peronu
- `stop_code` – publiczny kod przystanku (może być pusty dla peronów)
- `stop_name` – czytelna dla człowieka nazwa przystanku (np. `Wrocław Główny`, `Jelenia Góra`)
- `stop_desc` – opcjonalny tekstowy opis przystanku
- `stop_lat` – szerokość geograficzna przystanku w układzie współrzędnych WGS84
- `stop_lon` – długość geograficzna przystanku w układzie współrzędnych WGS84
- `location_type` – `0` = peron/przystanek, `1` = stacja nadrzędna
- `parent_station` – `stop_id` stacji nadrzędnej (puste dla samych stacji nadrzędnych)
- `platform_code` – oznaczenie peronu w obrębie stacji (np. `II`, `3`)

3.4 `trips.txt` – Definicje kursów (3 260 kursów)

Definiuje poszczególne kursy, tj. konkretne przejazdy pojazdu po danej trasie o określonym czasie. Każdy kurs należy do dokładnie jednej trasy i jednego wzorca kursowania (określającego dni, w których kurs jest realizowany).

- `route_id` – odwołanie do trasy, do której należy kurs (klucz obcy do `routes.txt`)
- `service_id` – odwołanie do kalendarza kursowania określającego, kiedy kurs jest realizowany (klucz obcy do `calendar.txt` / `calendar_dates.txt`)
- `trip_id` – unikalny identyfikator kursu
- `trip_headsign` – tekst wyświetlany na tablicy kierunkowej pojazdu (np. `Sędzisław`, `Wrocław Główny`)
- `direction_id` – binarny wskaźnik kierunku (`0` lub `1`) rozróżniający kursy w obu kierunkach na tej samej trasie
- `block_id` – identyfikuje blok pojazdu, umożliwiając powiązanie kolejnych kursów obsługiwanych przez ten sam pojazd

3.5 `stop_times.txt` – Czasy na przystankach (48 727 wpisów)

Największy plik w zbiorze danych. Zawiera czas przyjazdu i odjazdu każdego kursu na każdym obsługiwanym przystanku, definiując dokładną kolejność i rozkład czasowy przystanków w ramach każdego kursu. Jest to główny plik służący do konstrukcji grafu połączeń.

- `trip_id` – odwołanie do kursu (klucz obcy do `trips.txt`)
- `arrival_time` – planowany czas przyjazdu na przystanek, w formacie `HH:MM:SS`. Uwaga: wartości mogą przekraczać `23:59:59` dla kursów realizowanych po północy (np. `25:10:00` oznacza 01:10 następnego dnia)
- `departure_time` – planowany czas odjazdu z przystanku, w formacie `HH:MM:SS` (ta sama konwencja przekroczenia jak w `arrival_time`)
- `stop_id` – odwołanie do obsługiwanego przystanku (klucz obcy do `stops.txt`)
- `stop_sequence` – nieujemna liczba całkowita określająca kolejność przystanków w ramach kursu (zaczyna się od 0)
- `stop_headsign` – opcjonalne nadpisanie tablicy kierunkowej kursu na tym przystanku
- `pickup_type` – wskazuje, czy pasażerowie mogą wsiąść na tym przystanku: `0` = regularne wsiadanie, `1` = wsiadanie niedostępne
- `shape_dist_traveled` – odległość przebyta wzdłuż kształtu trasy do tego przystanku (może być puste)

3.6 `calendar.txt` – Kalendarze kursowania (3 260 wpisów)

Definiuje regularne tygodniowe wzorce kursowania. Każdy wpis określa, w które dni tygodnia dany `service_id` jest aktywny, wraz z okresem ważności (daty początkowa i końcowa w formacie `RRRRMMDD`).

- `service_id` – unikalny identyfikator wzorca kursowania
- `monday` ... `sunday` – flagi binarne (`0` lub `1`) wskazujące, czy kurs jest realizowany w danym dniu tygodnia
- `start_date` – pierwszy dzień okresu ważności kursowania, w formacie `RRRRMMDD`
- `end_date` – ostatni dzień okresu ważności kursowania, w formacie `RRRRMMDD`

3.7 `calendar_dates.txt` – Wyjątki w kursowaniu (5 549 wpisów)

Definiuje dodanie lub usunięcie kursów w konkretnych datach. Plik ten nadpisuje tygodniowe wzorce z `calendar.txt` w określone dni (np. święta państwowe, wydarzenia specjalne).

- `service_id` – odwołanie do modyfikowanego wzorca kursowania
- `date` – konkretna data wyjątku, w formacie `RRRRMMDD`
- `exception_type` – `1` = kurs *dodany* w tym dniu, `2` = kurs *usunięty* w tym dniu

3.8 `feed_info.txt` – Metadane kanału danych

Zawiera metadane dotyczące samego kanału GTFS, w tym wydawcę i okres ważności zbioru danych.

- `feed_publisher_name` – nazwa wydawcy kanału (`kiedyPrzyjedzie.pl`)
- `feed_publisher_url` – adres URL strony wydawcy
- `feed_lang` – język kanału (`pl`)
- `feed_start_date` – pierwszy dzień, dla którego dane kanału są ważne (`20260303`)
- `feed_end_date` – ostatni dzień, dla którego dane kanału są ważne (`20261212`)

3.9 Konstruowanie grafu połączeń

Aby wykorzystać dostarczone dane GTFS do wyszukiwania ścieżek, student musi skonstruować skierowany graf zależny od czasu. Poniższe wytyczne opisują wzajemne powiązania plików oraz sposób tworzenia krawędzi:

1. **Wierzchołki.** Każdy wierzchołek grafu reprezentuje przystanek (lub peron) zdefiniowany w `stops.txt`. Współrzędne geograficzne (`stop_lat`, `stop_lon`) mogą być wykorzystane do obliczenia heurystycznych estymacji odległości dla algorytmu A* (np. za pomocą wzoru haversine lub aproksymacji euklidesowej na rzutowanych współrzędnych).
2. **Krawędzie kursów (połączenia bezpośrednie).** Krawędź skierowana istnieje między dwoma kolejnymi przystankami w ramach tego samego kursu. Dla każdego kursu w `trips.txt` plik `stop_times.txt` dostarcza uporządkowaną sekwencję przystanków (posortowaną po `stop_sequence`). Dla kolejnych wpisów $(i, i+1)$ w tej sekwencji należy utworzyć krawędź od $\texttt{stop_id}_i$ do $\texttt{stop_id}_{i+1}$ z:
   - czas odjazdu = $\texttt{departure_time}_i$
   - czas przyjazdu = $\texttt{arrival_time}_{i+1}$
   - czas przejazdu = $\texttt{arrival_time}_{i+1} - \texttt{departure_time}_i$
   - nazwa linii = `route_short_name` (lub `route_long_name`, jeśli krótka nazwa jest pusta), uzyskana poprzez złączenie `trips.txt` $\rightarrow$ `routes.txt`
3. **Krawędzie przesiadkowe.** Gdy pasażer przesiada się z jednej linii na drugą na tej samej stacji, następuje przesiadka. Przesiadki powinny być modelowane przez łączenie przystanków, które mają tę samą `parent_station` (lub ten sam `stop_id`, jeśli hierarchia nadrzędny–podrzędny nie istnieje). Przesiadka wiąże się z czasem oczekiwania (różnica między czasem przyjazdu pierwszego kursu a czasem odjazdu następnego kursu na tej samej stacji) oraz zwiększa licznik przesiadek o jeden.
4. **Filtrowanie dni kursowania.** Nie wszystkie kursy realizowane są codziennie. Aby ustalić, czy kurs o danym `service_id` jest aktywny w określonym dniu, należy połączyć tygodniowy rozkład z `calendar.txt` z wyjątkami z `calendar_dates.txt`. Aplikacja studenta powinna przyjmować datę początkową (lub używać domyślnej daty w okresie ważności) i uwzględniać tylko kursy aktywne w danym dniu.

4 Zadania

1. Wykorzystując dostarczony zbiór danych GTFS od Kolei Dolnośląskich, zaimplementuj algorytm wyszukiwania najkrótszych ścieżek między podanymi przystankami `A` i `B`. Jako funkcję kosztu zastosuj (w zależności od decyzji użytkownika) czas przejazdu z `A` do `B` lub liczbę przesiadek.

   Aplikacja powinna przyjmować dane wejściowe w postaci 4 zmiennych:

   (a) przystanek początkowy `A`

   (b) przystanek końcowy `B`

   (c) kryterium optymalizacji: wartość t oznacza minimalizację czasu przejazdu, wartość p oznacza minimalizację liczby przesiadek

   (d) czas rozpoczęcia podróży

   Rozwiązanie powinno wypisywać na standardowe wyjście w kolejnych wierszach szczegółowe informacje o ścieżce, w tym przystanek początkowy, przystanek końcowy, nazwę wykorzystanej linii, czas rozpoczęcia, czas zakończenia, a na standardowe wyjście błędów wartość minimalizowanego kryterium oraz czas potrzebny do obliczenia najkrótszej ścieżki.

   Punktacja:

   (a) wyszukiwanie najkrótszej ścieżki z `A` do `B` za pomocą algorytmu Dijkstry, na podstawie kryterium czasu przejazdu (10 punktów).

   (b) wyszukiwanie najkrótszej ścieżki z `A` do `B` za pomocą algorytmu A*, na podstawie kryterium czasu przejazdu (25 punktów).

   (c) wyszukiwanie najkrótszej ścieżki z `A` do `B` za pomocą algorytmu A*, na podstawie kryterium liczby przesiadek (25 punktów).

   (d) modyfikacje algorytmu A* z (b) lub (c), pozwalające na skrócenie czasu obliczeń lub poprawę jakości znalezionego rozwiązania (10 punktów)

2. Wykorzystując dostarczony zbiór danych GTFS, zaimplementuj wyszukiwanie najkrótszej ścieżki z podanego przystanku `A`, odwiedzającej wszystkie przystanki z listy `L`$= A_2, \ldots, A_n$, i wracającej do `A`. Jako funkcję kosztu zastosuj (w zależności od decyzji użytkownika) czas przejazdu z `A` do `B` lub liczbę przesiadek.

   Aplikacja powinna przyjmować dane wejściowe składające się z 4 zmiennych:

   (a) przystanek początkowy `A`

   (b) lista przystanków do odwiedzenia oddzielona średnikiem `L`

   (c) kryterium optymalizacji: wartość t oznacza minimalizację czasu przejazdu, wartość p oznacza minimalizację liczby przesiadek

   (d) czas rozpoczęcia podróży

   Rozwiązanie powinno wypisywać na standardowe wyjście, w kolejnych wierszach, szczegółowe informacje o ścieżce, w tym przystanek początkowy, przystanek końcowy, nazwę wykorzystanej linii, czas rozpoczęcia, czas zakończenia, a na standardowe wyjście błędów wartość minimalizowanego kryterium oraz czas potrzebny do obliczenia najkrótszej ścieżki.

   Punktacja:

   (a) algorytm wyszukiwania najkrótszej ścieżki między wierzchołkami oparty na Tabu Search bez ograniczania rozmiaru T (10 punktów)

   (b) modyfikacja algorytmu (a) ze zmiennym rozmiarem T, zależnym od długości `L`, w celu minimalizacji funkcji kosztu (5 punktów)

(c) modyfikacja algorytmu (a) z dodaniem kryterium aspiracji, w celu minimalizacji funkcji kosztu (5 punktów)

(d) modyfikacja algorytmu (a) z dodaniem strategii próbkowania sąsiedztwa, w celu minimalizacji funkcji kosztu i skrócenia czasu obliczeń (10 punktów)

Do każdego z zadań przygotuj raport zawierający tło teoretyczne zastosowanych algorytmów, przykład użycia metody, opis dodanych modyfikacji do algorytmów bazowych oraz dodatkowe materiały wykorzystane do realizacji zadania, a także krótki opis użytych bibliotek. W podsumowaniu opisz problemy napotkane podczas implementacji. Raporty muszą zostać przesłane do prowadzącego co najmniej 24 godziny przed terminem oddania zadań.

5 Bibliografia

- GTFS Schedule Reference – oficjalna specyfikacja formatu General Transit Feed Specification
- Interaktywna animacja opisująca algorytm A*
- Prezentacja Forbota o algorytmie A*
- Wpis blogowy dotyczący A* i jego kroków
- Film opisujący algorytm A*
- Oryginalny artykuł o algorytmie A*
- E. Taillard – przegląd metody Tabu Search
- Knox – Tabu Search dla symetrycznego TSP
- Lai, Demirag, Leung – Tabu Search dla problemu trasowania pojazdów
- Silvestrin, Ritt – Tabu Search dla problemu trasowania pojazdów
- Soto i in. – Tabu Search dla problemu trasowania pojazdów
- Lin, Bian, Liu – hybryda Tabu Search i symulowanego wyżarzania z dynamicznym doborem sąsiedztwa dla TSP